Python Expressions and Evaluation

Lecture 3

CS 1001 Kirk Duran

Today

- Why programming languages let us work above machine-level details.
- How low-level and high-level languages differ.
- How compilation and interpretation connect source code to execution.
- Why Python is widely used and appropriate for this course.
- How Python evaluates one self-contained expression to produce one value.

Key idea

Today we evaluate expressions without introducing variables or persistent program state.

Part 1: Moving Away from Bare Metal

Programming Languages Bridge Human Ideas and Machine Execution

Definition

A programming language is a formal notation for expressing computations that a computer can execute.

- Its **syntax** specifies which forms are valid.
- Its **semantics** specifies what valid forms mean.
- Software translates or processes the notation so hardware can perform the requested computation.

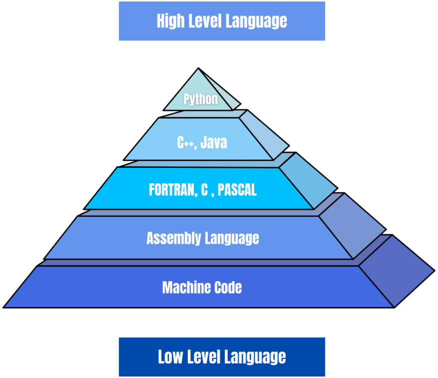
Low-Level Languages Stay Close to Hardware

Low-level languages

- Closely reflect processor operations.
- Expose memory and hardware details.
- Offer precise control but require more implementation detail.

Key idea

Low-level code gives the programmer more direct control over what the machine does, but that control comes with more machine-specific detail.



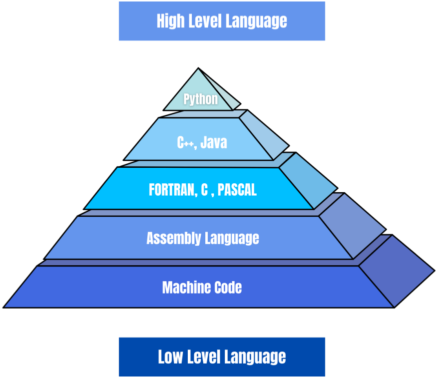
High-Level Languages Hide Machine Detail

High-level languages

- Express computations using human-oriented abstractions.
- Hide many processor-specific details.
- Improve readability, portability, and development speed.

Key idea

High-level languages let us write closer to the problem we are solving and farther from the processor instructions that eventually run.

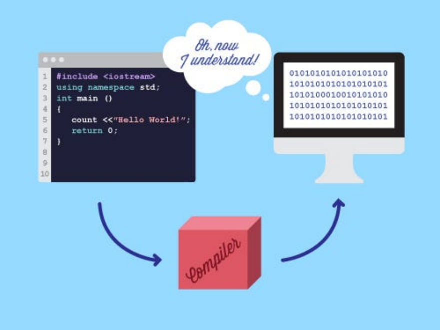


Compiled Languages Translate Before Execution

Compilation

A compiler translates source code into another executable form before that form runs.

- The compiler checks and translates source code ahead of execution.
- The output may be machine code, bytecode, or another lower-level form.
- The translated form can often run many times without recompiling from scratch.

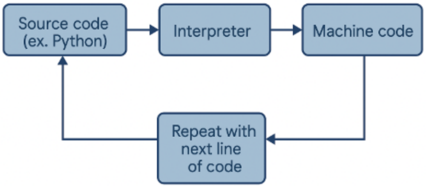


Interpreted Languages Process Code During Execution

Interpretation

An interpreter or runtime processes a program and performs its operations during execution.

- The runtime reads program forms and carries out their meaning as the program runs.
- This can support fast experimentation and interactive use.
- Many real systems blend strategies; CPython compiles source to bytecode and then executes that bytecode.



Key idea

Compiled and interpreted describe implementation strategies, not perfectly separate categories of languages.

Python Keeps Our Attention on the Computation

- Python is a high-level, general-purpose programming language.
- Its syntax emphasizes readability and supports rapid experimentation.
- It runs across major operating systems and has a large library ecosystem.
- It is used in automation, scientific computing, data analysis, artificial intelligence, web systems, and education.

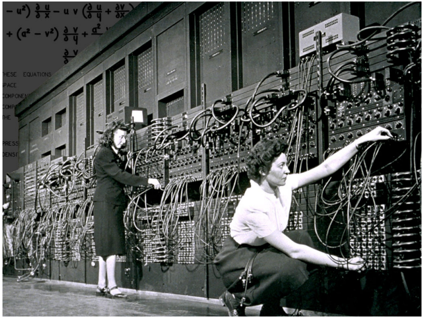
Example

In CPython, source code is compiled to bytecode, which the Python virtual machine executes.

Part 2: A Brief History of Programming Languages

Early Programming Stayed Close to the Machine

- Early electronic computers were programmed through switches, wiring, or numerical instruction codes.
- Machine language encoded processor instructions directly as numerical patterns.
- Assembly language replaced many numerical patterns with symbolic instruction names.
- Programs remained closely tied to a particular processor architecture.



Early High-Level Languages Described Problem Domains

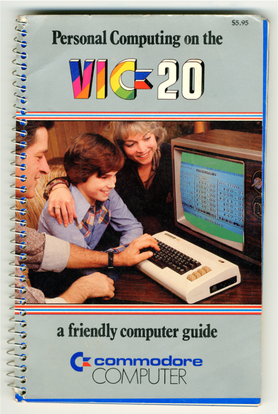
- **FORTRAN** supported scientific and numerical computing.
- **COBOL** supported business data processing.
- **Lisp** supported symbolic computation and early artificial intelligence research.
- Compilers translated these higher-level descriptions into lower-level instructions.

Key idea

Programmers increasingly described the problem to solve rather than every processor operation.

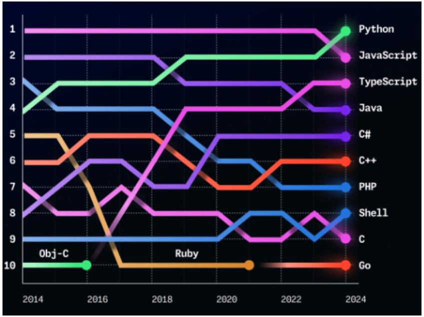
Languages Added Structure, Portability, and New Models

- **BASIC** made interactive programming more accessible.
- **C** combined portable systems programming with low-level control.
- **Pascal** emphasized structured programming and clear program organization.
- **Smalltalk** advanced object-oriented programming and graphical environments.



Modern Languages Grow Through Their Ecosystems

- Languages such as Python, Java, and JavaScript expanded with the internet and personal computing.
- Libraries package reusable solutions for common tasks.
- Development tools support testing, debugging, collaboration, and deployment.
- Communities, documentation, and application domains strongly influence adoption.



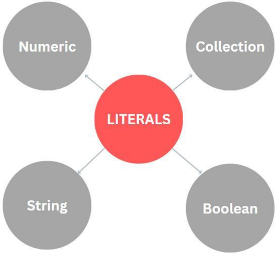
Part 3: Python Expressions Produce Values

A Literal Writes a Value Directly in Code

Definition

A **literal** is a value written directly in source code.

- `42`
- `3.14`
- `"hello"`
- `True`



Key idea

The written form represents a value; no name or storage location is needed for today's examples.

An Expression Evaluates to Produce One Value

Definition

An **expression** is code that Python evaluates to produce one value.

- `4 + 5` → `9`
- `3 * (8 - 2)` → `18`
- `"CS" + "1001"` → `"CS1001"`

Mental model

values

↓

operator

↓

new value

Operators Combine or Transform Values

- + addition
- − subtraction
- * multiplication
- / division
- // floor division
- % modulus
- ** exponentiation
- ( ) explicit grouping

Example

Each operator receives one or more values and produces a new value for the surrounding expression.

Python Evaluates Nested Expressions from the Inside Out

```
3 * (8 - 2)
```

1. The parentheses identify the inner expression: `8 - 2`.
2. That subexpression produces `6`.
3. The surrounding expression becomes `3 * 6`.
4. Multiplication produces the final value `18`.

Key idea

A completed subexpression becomes a value used by the surrounding operation.

Division Produces a Quotient

- `7 + 2` → `9`
- `7 - 2` → `5`
- `7 * 2` → `14`
- `7 / 2` → `3.5`

Example

`8 / 4` produces `2.0`, not `2`.

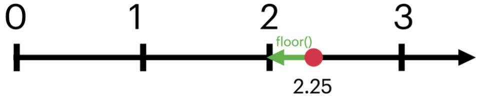
Key idea

In Python, the / operator produces a floating-point quotient, even when the division is exact.

Floor Division Counts Complete Groups

$$17\ //\ 5 \rightarrow 3$$

- Seventeen items contain three complete groups of five.
- The two remaining items do not form another complete group.
- For now, we use nonnegative values; negative operands require a more precise discussion of "floor."



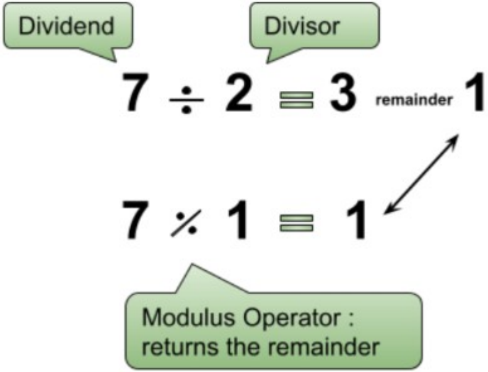
Modulus Produces the Remainder

$$17 \ \% \ 5 \rightarrow 2$$

- Three complete groups of five use fifteen items.
- Two items remain.
- Modulus is useful for cycles, divisibility, and extracting final digits.

Example

$123 \ \% \ 10 \rightarrow 3$



Exponentiation Repeats Multiplication

`2 ** 3` $\rightarrow$ 8

Interpretation

`2 ** 3` means three factors of two:

$$2 \times 2 \times 2 = 8$$

- Python uses `**` for exponentiation.
- The symbol `^` does not mean exponentiation in Python.
- Exponentiation has higher precedence than multiplication, division, addition, and subtraction.

Part 4: Order of Operations

Precedence Determines Which Operation Acts First

Priority	Operators	Rule
1	`( )`	Innermost grouping first
2	`**`	Right to left
3	`*`, `/`, `//`, `%`	Left to right
4	`+`, `-`	Left to right

Key idea

PEMDAS is a mnemonic. The table states the Python rules more precisely: multiplication and division share a level, as do addition and subtraction.

Associativity Resolves Operators at the Same Level

Usually left to right

`20 / 5 * 2`

`4.0 * 2` → `8.0`

Multiplication does not automatically occur before division; the two operators have equal precedence.

Exponentiation: right to left

`2 ** 3 ** 2`

`2 ** (3 ** 2)` → `512`

Exponentiation is the important exception in today's operator set.

Parentheses Override the Default Grouping

Default precedence

`2 + 3 * 4`

`2 + 12` $\rightarrow$ `14`

Explicit grouping

`(2 + 3) * 4`

`5 * 4` $\rightarrow$ `20`

Key idea

Parentheses can change a result and can make the intended grouping easier to read.

Reminder: The Processor Executes One Step at a Time

Von Neumann architecture

A stored-program computer keeps instructions and data in memory. The CPU repeatedly fetches instructions, decodes them, executes them, and stores results.

Processing cycle

Fetch an instruction from memory, decode the operation, execute it using CPU hardware, then move to the next instruction.

Key idea

Arithmetic operators eventually become processor operations carried out by the arithmetic logic unit (ALU).

Example

For today, ignore variables and stored program state. We are only tracing how one expression produces one value.

Part 5: Predict, Verify, and Explain

Predict Each Value Before Running Python

1. `2 + 3 * 4`
2. `(2 + 3) * 4`
3. `17 // 5`
4. `17 % 5`
5. `2 ** 3 ** 2`

In class

For each expression, record:

1. the final value,
2. the first operation performed, and
3. whether parentheses alter the result.

Use Colab to Test a Reasoned Prediction

1. Predict the final value.
2. Identify the expected evaluation order.
3. Enter one expression in one Colab cell.
4. Run the cell and compare the result.
5. Explain any difference between the prediction and Python's result.

Key idea

Colab verifies a prediction; it does not replace an explanation.

Evaluation and Display Are Different Actions

Evaluate an expression

```
2 + 3
```

The expression evaluates to 5. Colab automatically displays the value of the final expression in a cell.

Request display explicitly

```
print(2 + 3)
```

Python first evaluates `2 + 3`; then `print()` displays the resulting value.

Example

In a script, a bare expression usually produces no visible output. Evaluation still occurs.

One Expression Produces One Explainable Value

In class

Evaluate the following expression without running Python first:

```
17 % 5 + 2 ** 3 * 2
```

Report the final value and annotate the order of evaluation.

Key idea

Evaluate exponentiation first, then multiplication and division-level operators, then addition and subtraction. Parentheses override the default order.

Next Time

Naming and Retaining Values

Variables and Assignment